

Comprehensive Leave Management System

FIFO-Based Credit Deduction & Automated Utilization

Tech Stacks: Hono.js, Prisma, Bun, and TypeScript

System Overview

A robust REST API for managing employee Personal Time Off (PTO) with automated credit tracking.

Key Pillars:

1. **Posting:** Admin-driven allocation of leave hours.
2. **Utilization:** User-driven leave requests and approvals.
3. **Deduction:** Strict **FIFO (First-In, First-Out)** logic for consumption and expiration.

Tech Stacks: Hono server, Drizzle/Prisma ORM, Zod validation, and React frontend

Business Logic (The FIFO Principle)

Why FIFO? Ensures the oldest credits (those likely to expire first) are used before newer ones.

Consumption Logic: If a user needs 24 hours but has two 16-hour batches, the system fully depletes Batch A before touching Batch B.

Expiration Logic: System automatically clears unused hours from batches once the `expiresAt` date is reached.

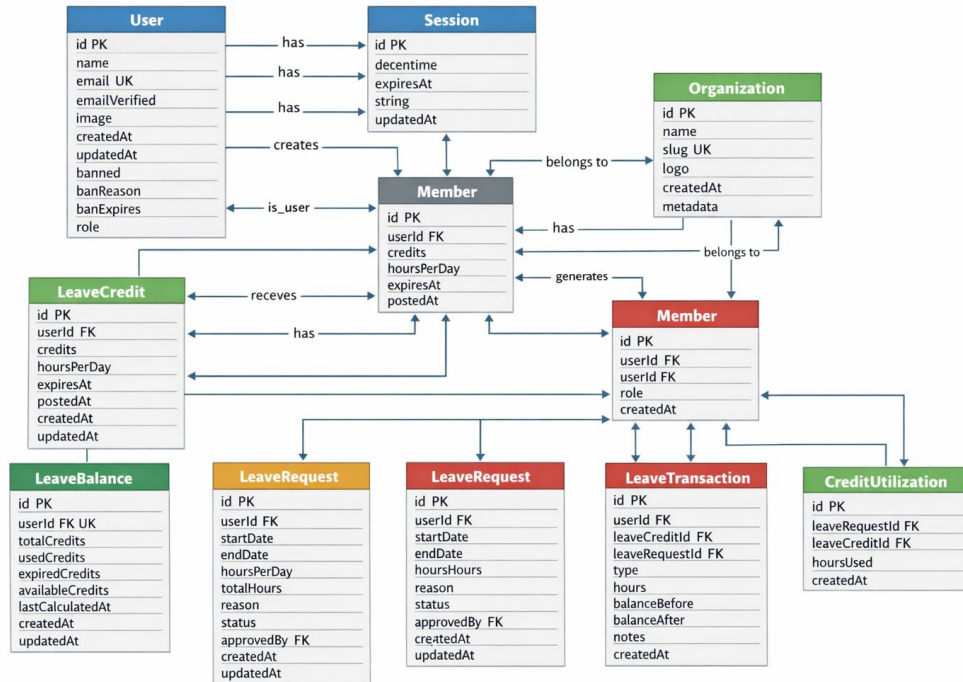
Data Architecture (ERD)

Core Entities:

- **LeaveCredit**: The source of hours (Batches).
- **LeaveBalance**: A real-time snapshot of user standing.
- **LeaveRequest**: The user's application for PTO.
- **CreditUtilization**: The "glue" table tracking which batch funded which request.
- **LeaveTransaction**: The immutable audit log.

availableCredits = totalCredits - usedCredits - expiredCredits

ERD Diagram



Admin Workflow

Credit Allocation: Admins can post hours to specific individuals or company-wide.

Management Tools:

- `POST /api/leave/credits`: Allocate hours with specific expiration dates.
- `GET /api/leave/admin/balances`: High-level view of all employee liabilities.
- `PATCH /api/leave/requests/:id`: Approve or reject pending requests with reason codes.

User Experience & Self-Service

Transparent Balances: Users see a breakdown of their credit batches and when they expire.

Request Flow:

- Validation checks for sufficient balance before submission.
- Calculates total hours based on date range and **hoursPerDay**.

Audit Trail: Users can view their own transaction history (CREDIT_POSTED vs. CREDIT_USED).

Automation & Cron Jobs

The Expiration Engine:

- Runs daily (e.g., `0 0 * * *`).
- Identifies batches past their `expiresAt` date.
- Converts remaining "Available" hours to "Expired" status.

Security: Cron endpoints are protected by a `CRON_SECRET` to prevent unauthorized triggers.

Error Handling & Validation

- **Insufficient Balance:** Returns a 400 Bad Request with details: *"Required: 24, Available: 8"*.
- **Authorization:** All routes protected by Better Auth middleware; roles differentiate User vs. Admin access.
- **Data Integrity:** Zod schemas ensure all dates and hour values are formatted correctly before hitting the database.